
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

⟨ School / Faculty placeholder ⟩

*drop
logo.png
here*

GoPlus

A Productivity Layer for Go

A source-language and Rust transpiler that keeps Go transparent

Author *⟨ Full name ⟩*
Student ID *⟨ ID ⟩*
Programme *⟨ Programme / Major ⟩*
Supervisor *⟨ Supervisor ⟩*
Course *⟨ Course / Subject ⟩*
Date *⟨ Month, Year ⟩*

Abstract

GoPlus is a surface language for Go, implemented as a transpiler that turns .gp sources into ordinary .go files. It preserves the entire Go ecosystem—toolchain, runtime, and packages—while sharpening the developer experience: concise error propagation, first-class enumerations with exhaustive pattern matching, and safe compile-time decorators. A central design constraint is that the generated Go stays readable and idiomatic, so teams retain full transparency and incur no lock-in. This report presents the motivation, design philosophy, language surface, compiler architecture, and developer tooling of GoPlus, and evaluates it against hand-written Go. On representative patterns the same behaviour is expressed in roughly 40–80% fewer tokens, while the generated code runs at parity with hand-written Go (measured at zero additional allocations), and transpilation completes in micro- to milliseconds. A small REST service written entirely in GoPlus demonstrates the approach on a medium-sized, real program.

Keywords: Go, transpiler, developer experience, error handling, algebraic enumerations, pattern matching, compile-time metaprogramming.

Contents

Abstract	i
1 Introduction and Motivation	1
2 Design Goals and Non-Goals	1
3 Language Overview	2
3.1 Concise error handling	2
3.2 Enumerations and exhaustive matching	2
3.3 Compile-time decorators	3
3.4 Derivations	3
3.5 Interoperability	4
3.6 The surface at a glance	4
4 Compiler Architecture	4
4.1 Semantic guarantees	5
4.2 Transparency, source maps, and incremental builds	6
4.3 Implementation footprint	6
5 Developer Tooling and Experience	6
5.1 Linting	7
5.2 Diagnostics	7
6 Evaluation	7

6.1	Token economy	8
6.2	Runtime parity	8
6.3	Decorators do real work	8
6.4	Transpile speed and scaling	9
6.5	Case study: a REST service	10
6.6	Threats to validity	10
7	Related Work and Positioning	10
8	Discussion, Limitations, and Adoption	11
8.1	What the design buys, and what it costs	11
8.2	Limitations	11
8.3	How a team would adopt it	11
9	Conclusion and Future Work	11
A	A complete example	13
B	Command-line reference	13

1 Introduction and Motivation

Go is prized for its simplicity, fast compilation, first-class concurrency, and an exceptional standard toolchain. That same minimalism, however, leaves recurring ergonomic gaps in everyday application code. Three stand out.

- **Verbose error flow.** The idiomatic `if err != nil` check, repeated after nearly every fallible call, dominates the visual weight of real code and obscures the intended logic.
- **No exhaustive enumerations.** A set of states is usually modelled with `const/iota` and a `switch`; the compiler cannot tell the developer when a new state is left unhandled.
- **No safe metaprogramming.** Cross-cutting concerns such as logging, retrying, or memoizing a function are written by hand at every call site, or bolted on with reflection.

GoPlus addresses these gaps with a *thin surface layer* that compiles down to ordinary Go. It is deliberately *not* a new language runtime and *not* a replacement for Go: it is a productivity layer whose output is plain, `gofmt`-formatted Go that any Go developer can read, review, and debug. A second, increasingly relevant motivation is the cost of writing code with AI assistants. Because a large-language-model’s effort and price scale with the number of tokens produced, a more concise yet unambiguous surface language lets an agent express the same behaviour with fewer tokens and fewer opportunities to get boilerplate wrong. Section 6 quantifies this effect.

Contributions. This report describes: (i) the design goals that keep GoPlus compatible and transparent; (ii) the language surface for error handling, enumerations and matching, decorators, and derivations; (iii) a straight-line compiler architecture that keeps the generated Go idiomatic; (iv) the developer tooling (formatter, linter, diagnostics, editor support); and (v) an evaluation against hand-written Go, including a medium-sized case study.

2 Design Goals and Non-Goals

GoPlus is defined as much by what it refuses to do as by what it adds. Keeping the scope small is what allows the generated Go to stay clean.

Goals	Non-goals
Practical compatibility with the Go toolchain	A borrow checker or ownership system
A low learning curve — standard Go syntax is also accepted	A free-form, token-level macro system
Concise error handling via <code>!</code> and <code>?</code>	A custom runtime replacing the Go runtime
Strong <code>enum</code> / <code>match</code> support with exhaustiveness	An overly complex type system that hurts simplicity
Flexible compile-time decorators, incl. user-defined ones	
Human-readable output, always <code>gofmt</code> -formatted	

Table 1: The scope of GoPlus, stated as goals and explicit non-goals.

The overarching principle is *transparency*: every construct must lower to Go that a person could plausibly have written by hand. This keeps the project free of lock-in—a team can, at any point, take the generated Go and walk away.

3 Language Overview

This section introduces the surface features at a conceptual level. The examples are illustrative; the point is the shape of the syntax and what it means, not any particular implementation detail.

3.1 Concise error handling

A function that can fail declares its result with a trailing `!` (`-> T!` for “a `T` or an error”, `-> !` for “possibly an error”). The postfix `?` operator propagates an error from a fallible call, so the happy path reads top to bottom.

Listing 1: Error propagation in GoPlus.

```
fn loadConfig(path: string) -> Config! {  
    raw    := readFile(path)?  
    parsed := parse(raw)?  
    return validate(parsed)?  
}
```

Conceptually, each `?` expands to the familiar Go check—capture the value and error, and return early on failure—so the emitted Go is exactly the code a developer would otherwise write, only generated automatically:

Listing 2: The Go that the previous example stands in for.

```
func loadConfig(path string) (Config, error) {  
    raw, err := readFile(path)  
    if err != nil { return Config{}, err }  
    parsed, err := parse(raw)  
    if err != nil { return Config{}, err }  
    return validate(parsed)  
}
```

The semantic layer only permits `?` inside a function whose result can carry an error, so misuse is caught at compile time rather than surfacing as a confusing Go error later.

3.2 Enumerations and exhaustive matching

GoPlus offers both simple enumerations and tagged (payload-carrying, optionally generic) enumerations, together with a `match` expression that the compiler checks for exhaustiveness. Forgetting a case is a compile error, so a newly added variant cannot silently slip through.

Listing 3: An enum state machine with an exhaustive match.

```
@derive(String)  
enum Status { Todo InProgress Done Cancelled }  
  
fn canAdvance(from: Status, to: Status) -> bool {  
    match from {  
        Todo      => to == Status::InProgress || to == Status::Cancelled  
        InProgress => to == Status::Done      || to == Status::Cancelled  
        Done       => false  
        Cancelled  => false  
    }  
}
```

A simple enum lowers to an integer type with named constants and a generated `String()` method; a `match` lowers to a Go `switch`. The difference from writing the `switch` by hand is that GoPlus *enforces* that every variant is handled—an assurance the Go compiler does not provide.

Tagged enumerations carry data and may be generic. They express a result or a sum type directly, and `match` binds the payload:

Listing 4: A generic tagged enumeration.

```
enum Result<T> {
    Ok(T)
    Err(string)
}

fn describe(r: Result<int>) -> string {
    match r {
        Ok(value) => fmt.Sprintf("ok: %d", value)
        Err(reason) => "error: " + reason
    }
}
```

A tagged enum lowers to a small tagged `struct` with constructor functions and Go generics, so it too remains ordinary, inspectable Go.

3.3 Compile-time decorators

Cross-cutting behaviour is attached with decorators. Built-in decorators cover common needs—`@log`, `@retry`, `@memoize`—and users can define their own following a small, Python-like contract (a decorator receives the next function in the chain and returns a wrapped function with the same signature).

Listing 5: Decorators replace hand-rolled wrappers.

```
@log
@retry(3, 10)
fn fetch(id: int) -> Payload! {
    return callService(id)?
}
```

Each decorator is expanded at compile time into a named wrapper function, so the resulting Go is ordinary, inspectable code—there is no reflection and no runtime indirection. The semantic layer validates that a custom decorator’s signature is compatible with the function it wraps.

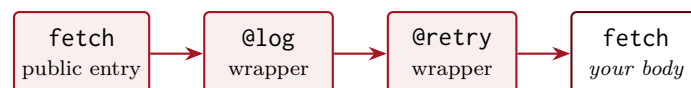


Figure 1: `@log @retry fn fetch` compiles to a chain of named wrapper functions: a call enters at the public `fetch` and unwinds inward to your body. The first decorator listed becomes the outermost wrapper.

3.4 Derivations

The `@derive` annotation generates routine methods so they need not be written and maintained by hand: `String`, `Debug`, `Equal`, `Clone`, and `JSON (de)serialization`.

Listing 6: Deriving boilerplate methods.

```
@derive(JsonMarshal, JsonUnmarshal, Clone)
struct Task {
    ID:    int    `json:"id"`
    Title: string `json:"title"`
    State: Status `json:"state"`
}
```

Notably, the derived `Clone` performs a *deep* copy of slice and map fields, avoiding a subtle and common Go bug in which a value copy silently shares the underlying storage. JSON derivations honour explicit field tags.

3.5 Interoperability

GoPlus is designed to sit inside real Go projects. Standard Go syntax is accepted alongside the GoPlus surface, a package may mix `.gp` and `.go` files that link together, and the standard library is used directly. Nothing about adopting GoPlus removes the option of dropping back to plain Go.

3.6 The surface at a glance

Table 2 summarises how each concern is expressed in GoPlus versus plain Go. In every row the GoPlus form is not only shorter but also carries a guarantee the plain-Go form lacks.

Concern	GoPlus	Plain Go
Error propagation	<code>x := f()? in a $\rightarrow T!$ function</code>	<code>x, err := f(); if err != nil { return }</code> at every call
Finite state set	<code>enum + exhaustive match</code> (compiler-checked)	<code>const/iota + switch</code> (unchecked)
Cross-cutting behaviour	a decorator, e.g. <code>@retry(3)</code>	a hand-written wrapper function
Routine methods	<code>@derive(String, Equal, Clone, JSON)</code>	written and maintained by hand
Safe deep copy	<code>@derive(Clone)</code> slices/maps	a value copy silently shares backing storage

Table 2: Each concern, expressed in GoPlus and in plain Go.

4 Compiler Architecture

GoPlus is a straight-line transpiler. Each stage has a single responsibility, and the last two stages hand off to the unmodified Go toolchain.

Lexer

converts source into a token stream.

Parser

builds an abstract syntax tree. Language constructs (declarations, `if/else`, `match`, and the control-flow statements `for/switch/select`) are parsed into structured nodes; a few line-level

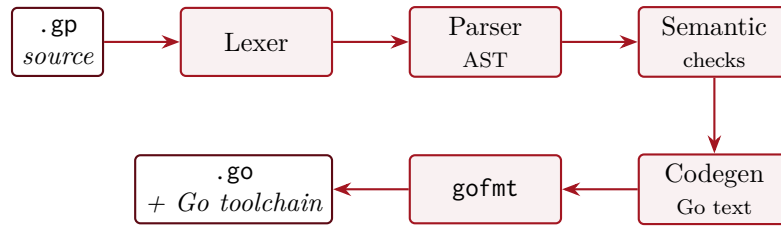


Figure 2: The GoPlus pipeline: lex → parse → analyze → generate → format → Go toolchain.

constructs are kept as verbatim pass-through. Unfamiliar shapes fall back to raw text rather than failing the whole file.

Semantic layer

enforces the guarantees that make GoPlus safe: exhaustive `match`, valid `?` context, decorator contracts, and duplicate/collision checks. It descends into every block—including the bodies of loops and switches—so nested constructs are analyzed at any depth.

Codegen

emits deliberately readable Go: enums become integer or tagged struct types, `match` becomes a `switch`, `?` becomes an explicit error check, and `@derive` emits the corresponding methods.

gofmt

formats the generated Go, so the output reads like code a person wrote.

Figure 3 sketches how three surface constructs are lowered. In every case the result is plain Go with no hidden machinery.

GoPlus construct	generated Go (in essence)
<code>x := f()?</code>	<code>⇒ x, err := f()</code> <code>if err != nil { return z, err }</code>
<code>match s { A => e }</code>	<code>⇒ switch s { case A: return e }</code>
<code>@memoize fn g(n)</code>	<code>⇒ g_inner + guarded cache + g</code>

Figure 3: Representative desugarings. Each surface construct lowers to plain, idiomatic Go; nothing is added at run time beyond what the construct means.

4.1 Semantic guarantees

The value of the semantic layer is that it turns whole classes of latent bugs into compile-time errors, close to where the developer is working rather than deep inside a generated Go file:

- **Exhaustiveness.** A `match` over an enumeration must handle every variant (or use an explicit wildcard); adding a variant makes any incomplete match fail to compile.
- **Error context.** The `?` operator is only accepted in a function whose result can carry an error, so it can never expand into invalid Go.
- **Decorator contracts.** A custom decorator is checked against the signature of the function it wraps, on both free functions and methods.
- **Name integrity.** Duplicate declarations and collisions with the compiler’s generated names are detected before code generation.

Because analysis descends into every nested block, these checks hold at any depth—for example, for a `match` written inside a loop or a switch case.

4.2 Transparency, source maps, and incremental builds

Two properties keep GoPlus practical at scale. First, the generated Go is always run through `gofmt` and is written to read like hand-authored code, so it can be reviewed, stepped through in a debugger, and—if ever necessary—kept as the project’s own source. Second, two facilities support tooling: *source maps* record `.gp` ↔ `.go` ranges for declarations, functions, match arms, and statements, enabling bidirectional “go to definition”; and a *content-hash package cache* lets incremental builds skip packages whose sources have not changed, keeping continuous-integration loops fast.



Figure 4: A source map records `.gp` ↔ `.go` ranges, so an editor can jump in either direction—from a GoPlus construct to the Go it produced, and back.

4.3 Implementation footprint

The compiler is a modestly-sized Rust codebase—roughly nine thousand lines of implementation—partitioned so that no component grows into a catch-all. Figure 5 shows the distribution; code generation is the largest, as befits the stage that must emit idiomatic Go for every construct. The project is backed by 80 unit and integration tests and 18 runnable examples.

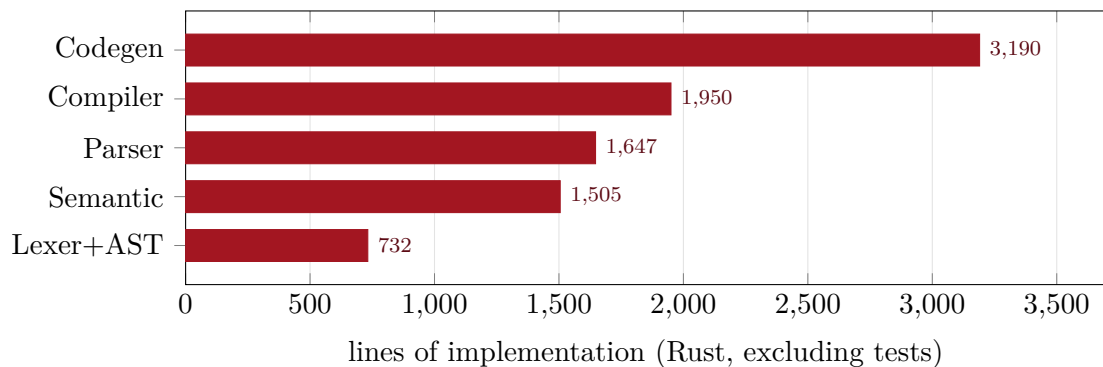


Figure 5: Implementation size by component. The frontend, semantic layer, code generator, and project orchestration are cleanly separated.

5 Developer Tooling and Experience

A language layer is only as good as the tooling around it. GoPlus ships a coherent set of developer tools built on the same frontend.

- **Formatter.** A rewriting formatter reproduces a canonical form of each source file. It is *comment-preserving* and *idempotent*, and it is safe by construction: rather than risk dropping a comment, it declines to overwrite a file it cannot format losslessly.

- **Linters.** A set of stably-coded rules flag stylistic and suspicious-code issues without generating Go—for example, an unreachable arm after a wildcard, or a wildcard that quietly defeats the exhaustiveness of an enum match.
- **Diagnostics.** Errors and warnings carry a stable code, a precise caret span, and an actionable hint, and are available in a machine-readable form for editors.
- **Editor support.** A first-party editor extension provides syntax highlighting, inline diagnostics, hover documentation, snippets, format-on-save, and source navigation between `.gp` and the generated `.go`.

5.1 Linting

The linter reports issues that the type system alone would not, each under a stable code so they can be tracked and suppressed deliberately. Table 3 lists the current rules.

Code	Detects
L0001	An imported package that is never used
L0002	A type or function name that breaks the naming convention
L0003	An empty function body
L0004	A redundant trailing <code>return</code> in a void function
L0005	An unreachable match arm following a wildcard
L0006	A function large enough to warrant refactoring
L0007	An enum used in formatting without a derived <code>String</code>
L0008	A wildcard that hides unlisted enum variants, defeating exhaustiveness

Table 3: Linter rules, each with a stable diagnostic code.

5.2 Diagnostics

Every diagnostic carries a severity, a stable code, a precise caret span, and an actionable hint. A human-readable rendering looks like the following, and the same information is available as structured JSON for editors and other tools:

Listing 7: A diagnostic with code, span, and hint.

```
warning L0008: `_` in this match on enum `Color` hides 2 unlisted
variant(s): Green, Blue
match c {
    ^^^^^^^
hint: list the variants explicitly so exhaustiveness catches new ones
```

6 Evaluation

Three small, reproducible benchmarks test the project’s central claims: that GoPlus reduces the amount of code written, that it adds no runtime cost, and that it transpiles quickly. Each measures the *same behaviour* implemented two ways where a comparison applies.

6.1 Token economy

For five representative patterns, the same program was written once in GoPlus and once in plain hand-written Go. Token counts use a modern LLM tokenizer.

Pattern	GoPlus (tok.)	Hand Go (tok.)	Reduction
Error-propagation chain (?)	81	130	−38%
Enum state machine + <code>String</code>	69	151	−54%
Struct equality/debug derive	30	94	−68%
JSON model derive	61	244	−75%
Decorated API call (retry + log)	40	188	−79%
Total	281	807	−65%

Table 4: The same behaviour uses substantially fewer tokens in GoPlus. The saving grows with the amount of boilerplate replaced—derivations and decorators, which stand in for whole method bodies and wrappers, save the most.

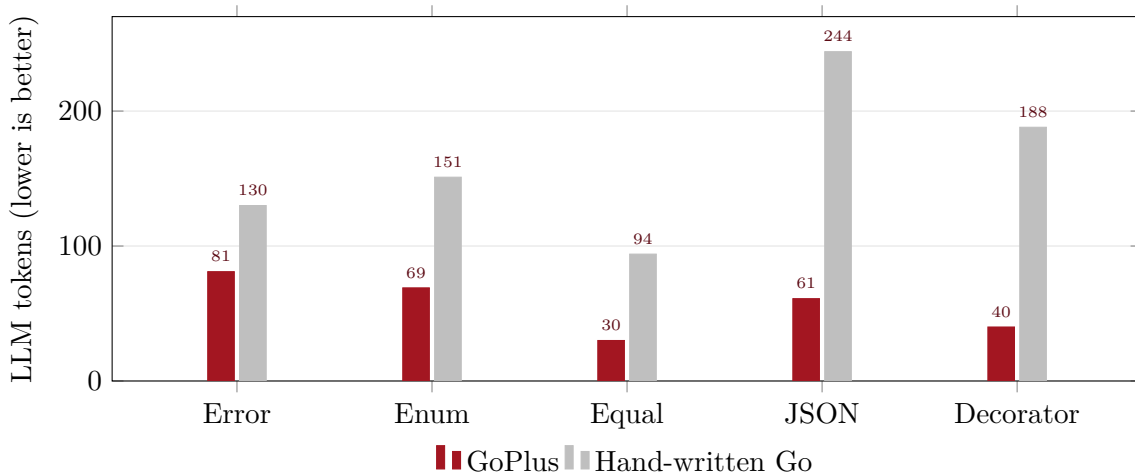


Figure 6: Tokens for the same behaviour across five patterns: GoPlus (red) versus hand-written Go (grey). The gap widens as more boilerplate is replaced.

Beyond the raw count, the GoPlus versions carry stronger guarantees: the `enum match` is exhaustiveness-checked, whereas the equivalent hand-written Go `switch` is not.

6.2 Runtime parity

Because GoPlus adds no runtime layer, its output should run exactly as fast as hand-written Go. The same algorithm was benchmarked as GoPlus-generated Go and as a hand-written Go equivalent, in the same package, after asserting that both produce identical results.

6.3 Decorators do real work

The measurements so far concern GoPlus’s *sugar*, which should cost nothing. Decorators are the opposite case: they exist to add behaviour and should add exactly as much as that behaviour requires—no more. Figure 7 shows `@memoize` applied to a naive recursive Fibonacci: a repeated query is answered from the cache in a few nanoseconds, versus roughly nine milliseconds for the

Implementation	Time / op (median)	Allocations / op
GoPlus-generated Go	$\approx 126 \mu\text{s}$	0
Hand-written Go	$\approx 128 \mu\text{s}$	0

Table 5: The two implementations are within run-to-run noise of each other, both with zero allocations. The enum/match sugar desugars to a plain integer switch.

un-memoized recursion—a speed-up of over a million times. The point is not the speed-up but that a single annotation delivers the cache one would otherwise write, test, and maintain by hand.

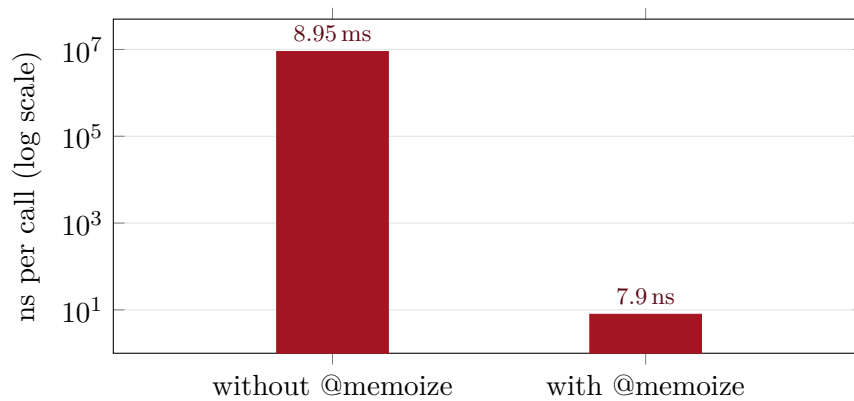


Figure 7: A repeated `fib(32)` query: without `@memoize` the naive recursion takes ≈ 9 ms; `@memoize` answers from its cache in ≈ 8 ns. Note the logarithmic scale.

6.4 Transpile speed and scaling

Transpilation must not slow a build or CI loop. On a sample program the frontend is microsecond-fast and code generation low-millisecond (parsing $\approx 3 \mu\text{s}$, analysis $\approx 6 \mu\text{s}$, code generation ≈ 1.4 ms). More importantly, end-to-end transpile time scales linearly with program size: Figure 8 plots wall-clock time against the number of functions, and even an 800-function source transpiles in well under a tenth of a second. A content-hash package cache further skips packages whose sources have not changed on rebuild.

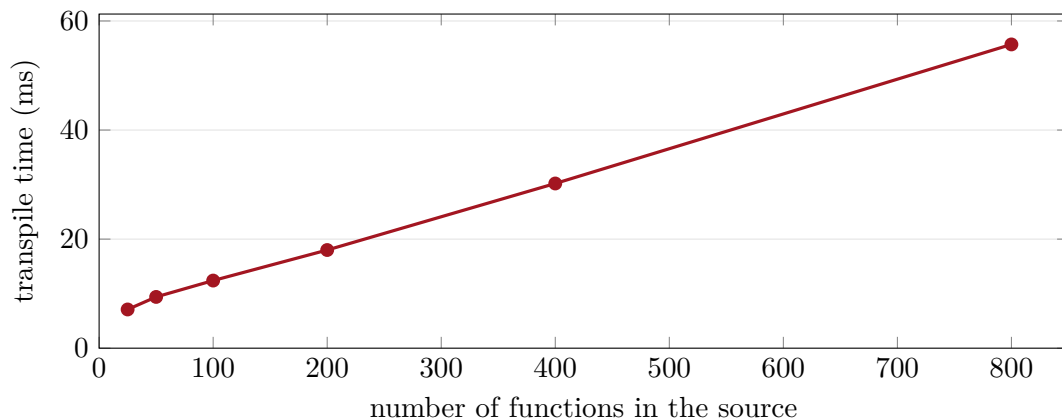


Figure 8: End-to-end transpile time grows linearly with program size; an 800-function source transpiles in ≈ 56 ms.

6.5 Case study: a REST service

To test the “readable, debuggable, review-friendly” promise on something past a single-file example, a small task-tracker REST service was written entirely in GoPlus: HTTP handlers, a service layer, an in-memory store, an enum-based state machine, and JSON serialization. The service runs, and its handlers are covered by tests driven through the standard HTTP test harness.

The design follows a conventional three-layer shape—handlers translate HTTP to calls, a service layer holds the business rules, and a store persists data—and each GoPlus feature lands where it is most useful. The store returns independent deep copies via a derived Clone, so a handler cannot accidentally mutate shared state. The service validates every state change through a single exhaustive match, so an illegal transition is rejected uniformly. Error handling threads through the layers with `?`, and a logging concern is a single decorator rather than repeated boilerplate. Request and response payloads (de)serialize through derived JSON methods. The store is an in-memory map behind a small set of functions; a real database driver would slot in at exactly that boundary without touching the handlers or the service.

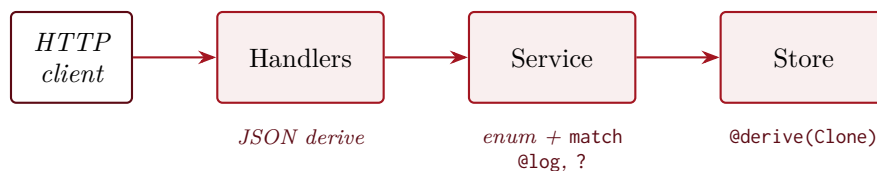


Figure 9: The case-study service is layered, and each GoPlus feature lands where it is most useful. A real database driver would replace the in-memory store without touching the layers above it.

Two observations are worth recording. First, the layered design reads cleanly: error flow threads through the layers with `?`, the state machine is a single exhaustive match, and cross-cutting logging is one decorator. Second—and more tellingly—building a medium-sized program exercised code paths that single-file examples never had, and *surfaced real defects in the generated code that were then fixed*. That a case study earns its keep by finding bugs is itself evidence that the approach scales beyond toy inputs.

6.6 Threats to validity

These are small, illustrative inputs measured on a single machine; the magnitudes should be read as indicative rather than guaranteed. Token counts depend on the chosen tokenizer and vary by model. The runtime-parity benchmark deliberately isolates language *sugar* (no decorators) to measure overhead fairly; decorators, by design, add exactly the work they describe (a cache, a retry loop), which is code one would otherwise write by hand.

7 Related Work and Positioning

GoPlus occupies a familiar niche: a surface language that compiles to a mainstream target while keeping the output transparent. The clearest analogy is the way a typed surface language can compile to a widely deployed dynamic language, adding ergonomics and static checks without introducing a new runtime; GoPlus applies the same philosophy to Go.

Its individual features are well established in language design—error propagation operators, algebraic enumerations with exhaustive matching, and compile-time decorators all have mature precedents. GoPlus’s contribution is not to invent these mechanisms but to *compose a carefully scoped subset of them on top of Go while guaranteeing readable, idiomatic Go output*. In contrast to approaches that introduce a bespoke runtime, a heavyweight type system, or opaque macro

expansion, GoPlus keeps the generated code inspectable and the language small, which is precisely what preserves compatibility and avoids lock-in.

8 Discussion, Limitations, and Adoption

8.1 What the design buys, and what it costs

The consistent theme across the evaluation is that GoPlus removes repetitive, error-prone code while adding static guarantees, at no runtime cost. The cost it does impose is a modest one: a team takes on a build step and a surface syntax that, while close to Go, is not Go. GoPlus keeps that cost low by two deliberate choices—accepting standard Go syntax alongside its own, and emitting readable Go—so the barrier to trying it, and the barrier to leaving it, are both small.

8.2 Limitations

GoPlus is an evolving project, and several boundaries are worth stating plainly. A handful of line-level constructs are still carried through as verbatim text rather than fully structured, which bounds how deeply they can be analyzed. Package-qualified decorators are not signature-checked. The richer derivations (ordering, hashing) and user-defined derivations are not yet available. And the evaluation, while measured on real inputs, uses small programs on a single machine; the numbers should be read as indicative of direction and magnitude rather than as guarantees.

8.3 How a team would adopt it

Because a package may mix `.gp` and `.go` files, adoption can be incremental: a team can write one new package—or one new file—in GoPlus, review the generated Go, and expand from there. The generated code is checked in or produced by the build like any other artefact, and the standard Go tooling (testing, profiling, modules) applies unchanged to it. If the experiment does not work out, the generated Go is the exit path.

9 Conclusion and Future Work

GoPlus is a productivity layer that makes everyday Go more concise and safer to write while keeping the generated Go fully transparent. It streamlines error flow, adds exhaustiveness-checked enumerations and pattern matching, and offers safe compile-time decorators and derivations—backed by a coherent toolchain of formatter, linter, diagnostics, and editor support. The evaluation indicates a substantial reduction in the amount of code written (about 65% fewer tokens on the measured patterns) with no measurable runtime cost and fast transpilation, and a medium-sized case study shows the approach holds up on a real service.

Several directions remain open. On the language and codegen side: richer derivations (ordering and hashing), user-defined derivations, and deeper structured parsing of the remaining line-level constructs. On the ecosystem side: better large-project package handling, wider editor and marketplace availability, and a broader benchmark suite tracked over time. The guiding constraint will not change: whatever is added must still lower to Go that a person could have written by hand.

References

- [1] The Go Programming Language. <https://go.dev>.
- [2] `gofmt` — format Go source code. <https://pkg.go.dev/cmd/gofmt>.

- [3] *Effective Go*. https://go.dev/doc/effective_go.
- [4] The Rust Programming Language (implementation language of the transpiler). <https://www.rust-lang.org>.
- [5] mdBook — documentation tooling. <https://rust-lang.github.io/mdBook>.
- [6] GoPlus source repository. <https://github.com/Trinq2003/golang-plus>.

A A complete example

The following self-contained program exercises the core surface features: a derived enumeration, an exhaustive `match`, error propagation with `?`, and a structured loop. It is ordinary GoPlus that compiles to plain, `gofmt`-formatted Go.

Listing 8: A small, complete GoPlus program.

```
package main

import "fmt"

@derive(String)
enum Band { Low Medium High }

fn classify(score: int) -> Band! {
  if score < 0 {
    return error("score must be non-negative")
  }
  if score < 40 {
    return Band::Low
  }
  if score < 80 {
    return Band::Medium
  }
  return Band::High
}

fn label(b: Band) -> string {
  match b {
    Low    => "needs work"
    Medium => "acceptable"
    High   => "excellent"
  }
}

fn main() -> ! {
  for _, s := range []int{12, 55, 93} {
    band := classify(s)?
    fmt.Println(s, band, label(band))
  }
  return
}
```

B Command-line reference

Every command accepts a `.gp` file or a package directory.

Command	Purpose
check	Parse, analyze, transpile, and type-check via the Go compiler.
transpile	Emit Go (optionally with a <code>.gp</code> $\leftrightarrow$ <code>.go</code> source map).
build	Transpile, then build a binary.
run	Transpile, then run.
test	Transpile, link sibling Go and test files, then run <code>go test</code> .
fmt	Format <code>.gp</code> in place (comment-safe), with <code>-check</code> / <code>-stdout</code> modes.
lint	Report style and suspicious-code diagnostics without generating Go.
navigate	Resolve a position between <code>.gp</code> and generated <code>.go</code> via a source map.

Table 6: The GoPlus command-line interface.